

解题能力训练-DP

清华大学计算机科学与技术系 顾霆枫

DP问题一页纸

- ▶ Step1 划分阶段，寻找决策，确定状态
 - ▶ 将影响后续决策的因素计入状态
 - ▶ 优化掉不影响后续决策的因素
- ▶ Step2 明确转移，写出转移关系
 - ▶ 对每个状态，写出每种决策使其转移到哪种状态
 - ▶ 写出决策对目标值的影响
- ▶ Step3 倒推状态转移方程
 - ▶ 把转移箭头右侧的下标统一
 - ▶ 合并转移式
- ▶ Step4 确定DP顺序，**确定边界情况，写出朴素DP**
- ▶ (Step5) 添加优化

课程计划

上午讲历年联赛的一些DP题

下午讲上午没讲完的部分，再补充一些有趣的题目

NOIP2018 保卫王国

题目描述

展开

Z 国有 n 座城市， $(n - 1)$ 条双向道路，每条双向道路连接两座城市，且任意两座城市都能通过若干条道路相互到达。

Z 国的国防部长小 Z 要在城市中驻扎军队。驻扎军队需要满足如下几个条件：

- 一座城市可以驻扎一支军队，也可以不驻扎军队。
- 由道路直接连接的两座城市中至少要有一座城市驻扎军队。
- 在城市里驻扎军队会产生花费，在编号为 i 的城市中驻扎军队的花费是 p_i 。

小 Z 很快就规划出了一种驻扎军队的方案，使总花费最小。但是国王又给小 Z 提出了 m 个要求，每个要求规定了其中两座城市是否驻扎军队。小 Z 需要针对每个要求逐一给出回答。具体而言，如果国王提出的第 j 个要求能够满足上述驻扎条件（不需要考虑第 j 个要求之外的其它要求），则需要给出在此要求前提下驻扎军队的最小开销。如果国王提出的第 j 个要求无法满足，则需要输出 -1 。现在请你来帮助小 Z。

数据类型的含义：

- **A**：城市 i 与城市 $i + 1$ 直接相连。
- **B**：任意城市与城市 1 的距离不超过 100（距离定义为最短路径上边的数量），即如果这棵树以 1 号城市为根，深度不超过 100。
- **C**：在树的形态上无特殊约束。
- **1**：询问时保证 $a = 1, x = 1$ ，即要求在城市 1 驻军。对 b, y 没有限制。
- **2**：询问时保证 a, b 是相邻的（由一条道路直接连通）
- **3**：在询问上无特殊约束。

对于 100% 的数据，保证 $1 \leq n, m \leq 10^5$ ， $1 \leq p_i \leq 10^5$ ， $1 \leq u, v, a, b \leq n$ ， $a \neq b$ ， $x, y \in \{0, 1\}$ 。

数据规模与约定

测试点编号	type	$n = m =$
1, 2	A3	10
3, 4	C3	10
5, 6	A3	100
7	C3	100
8, 9	A3	2×10^3
10, 11	C3	2×10^3
12, 13	A1	10^5
14, 15, 16	A2	10^5
17	A3	10^5
18, 19	B1	10^5
20, 21	C1	10^5
22	C2	10^5
23, 24, 25	C3	10^5

NOIP2018 保卫王国

- ▶ 简化题意：
- ▶ 给出一棵树，从树上选点，一个合法的选点方案要保证任意一条边至少有一个端点被选，选点方案的代价为所选点权之和
- ▶ 每次给出一个询问，询问给定两个点选或不选的状态下，合法选点方案的最小代价

NOIP2018 保卫王国

- ▶ 简化题意：
- ▶ 给出一棵树，从树上选点，一个合法的选点方案要保证任意一条边至少有一个端点被选，选点方案的代价为所选点权之和
- ▶ 每次给出一个询问，询问给定两个点选或不选的状态下，合法选点方案的最小代价
- ▶ 首先考虑暴力，假设我允许你 $O(nm)$ 地做，那么显然是每次做一遍树形dp

NOIP2018 保卫王国

- ▶ 简化题意：
- ▶ 给出一棵树，从树上选点，一个合法的选点方案要保证任意一条边至少有一个端点被选，选点方案的代价为所选点权之和
- ▶ 每次给出一个询问，询问给定两个点选或不选的状态下，合法选点方案的最小代价
- ▶ 首先考虑暴力，假设我允许你 $O(nm)$ 地做，那么显然是每次做一遍树形dp
- ▶ 用 $f[u][0/1]$ 表示以 u 为根的子树，满足要求的情况下， u 不选/选的最小代价

NOIP2018 保卫王国

- ▶ 简化题意:
- ▶ 给出一棵树，从树上选点，一个合法的选点方案要保证任意一条边至少有一个端点被选，选点方案的代价为所选点权之和
- ▶ 每次给出一个询问，询问给定两个点选或不选的状态下，合法选点方案的最小代价
- ▶ 首先考虑暴力，假设我允许你 $O(nm)$ 地做，那么显然是每次做一遍树形dp
- ▶ 用 $f[u][0/1]$ 表示以 u 为根的子树，满足要求的情况下， u 不选/选的最小代价
- ▶ 转移应该比较好想，枚举 u 的儿子 v
- ▶ $f[u][0] = \sum f[v][1]$; $f[u][1] = p[u] + \sum \min(f[v][0], f[v][1])$

NOIP2018 保卫王国

- ▶ 简化题意:
- ▶ 给出一棵树，从树上选点，一个合法的选点方案要保证任意一条边至少有一个端点被选，选点方案的代价为所选点权之和
- ▶ 每次给出一个询问，询问给定两个点选或不选的状态下，合法选点方案的最小代价
- ▶ 首先考虑暴力，假设我允许你 $O(nm)$ 地做，那么显然是每次做一遍树形dp
- ▶ 用 $f[u][0/1]$ 表示以 u 为根的子树，满足要求的情况下， u 不选/选的最小代价
- ▶ 转移应该比较好想，枚举 u 的儿子 v
- ▶ $f[u][0] = \sum f[v][1]$; $f[u][1] = p[u] + \sum \min(f[v][0], f[v][1])$
- ▶ 在处理到给定的 a/b 点时，强制把不合法的 f 值设置为 INF 即可

NOIP2018 保卫王国

- ▶ 简化题意:
- ▶ 给出一棵树，从树上选点，一个合法的选点方案要保证任意一条边至少有一个端点被选，选点方案的代价为所选点权之和
- ▶ 每次给出一个询问，询问给定两个点选或不选的状态下，合法选点方案的最小代价
- ▶ 首先考虑暴力，假设我允许你 $O(nm)$ 地做，那么显然是每次做一遍树形dp
- ▶ 用 $f[u][0/1]$ 表示以 u 为根的子树，满足要求的情况下， u 不选/选的最小代价
- ▶ 转移应该比较好想，枚举 u 的儿子 v
- ▶ $f[u][0] = \sum f[v][1]$; $f[u][1] = p[u] + \sum \min(f[v][0], f[v][1])$
- ▶ 在处理到给定的 a/b 点时，强制把不合法的 f 值设置为 INF 即可
- ▶ 写出来就是44分

NOIP2018 保卫王国

- ▶ 那么考虑这个做法低效在哪里

NOIP2018 保卫王国

- ▶ 那么考虑这个做法低效在哪里
- ▶ 我们每一次只会对2个点的情况进行限制，而大量不在a~b路径上的dp转移被重复做了很多次
- ▶ 我们希望通过某种方式，快速把已经dp过一遍的信息重新利用上

NOIP2018 保卫王国

- ▶ 那么考虑这个做法低效在哪里
- ▶ 我们每一次只会对2个点的情况进行限制，而大量不在a~b路径上的dp转移被重复做了很多次
- ▶ 我们希望通过某种方式，快速把已经dp过一遍的信息重新利用上
- ▶ 我们需要注意到这样一个性质，给定树上的一条u~t的祖先链，t是u的祖先，在给定了u和t选或不选的状态后，u~t的链以及中间节点的所有分支的决策都是确定的，且不会受到（u的子树） $\cup$ （t子树以外部分）的决策的影响。
- ▶ 然后树链剖分（划掉

NOIP2018 保卫王国

- ▶ 那么考虑这个做法低效在哪里
- ▶ 我们每一次只会对2个点的情况进行限制，而大量不在a~b路径上的dp转移被重复做了很多次
- ▶ 我们希望通过某种方式，快速把已经dp过一遍的信息重新利用上
- ▶ 我们需要注意到这样一个性质，给定树上的一条u~t的祖先链，t是u的祖先，在给定了u和t选或不选的状态后，u~t的链以及中间节点的所有分支的决策都是确定的，且不会受到（u的子树） $\cup$ （t子树以外部分）的决策的影响。
- ▶ 然后树链剖分（划掉
- ▶ 树链剖分感觉确实是可以做的，但是想了想不好写，我就没有仔细研究

NOIP2018 保卫王国

- ▶ 都想到树链剖分了，事实上，在这种不带修改只查询的情况下，很多时候树链剖分都能被倍增取代
- ▶ 提示到这里了，可以再想一下

NOIP2018 保卫王国

- ▶ 都想到树链剖分了，事实上，在这种不带修改只查询的情况下，很多时候树链剖分都能被倍增取代
- ▶ 提示到这里了，可以再想一下
- ▶ 记 $g[u][i][ku=0/1][kt=0/1] = f[fa(u,i)][kt] - f[u][ku]$
- ▶ 其中 $fa(u,i)$ 表示深度比 u 小 2^i 的祖先
- ▶ 这个倍增数组可以轻松预处理

NOIP2018 保卫王国

- ▶ 都想到树链剖分了，事实上，在这种不带修改只查询的情况下，很多时候树链剖分都能被倍增取代
- ▶ 提示到这里了，可以再想一下
- ▶ 记 $g[u][i][ku=0/1][kt=0/1] = f[fa(u,i)][kt] - f[u][ku]$
- ▶ 其中 $fa(u,i)$ 表示深度比 u 小 2^i 的祖先
- ▶ 这个倍增数组可以轻松预处理
- ▶ 那么每次拿到 a, b ，求出 LCA c ，通过倍增的方式求出 a 和 b 到 c 的两个儿子这条链上满足要求的最小代价，倍增的细节画图解释

NOIP2018 保卫王国

- ▶ 都想到树链剖分了，事实上，在这种不带修改只查询的情况下，很多时候树链剖分都能被倍增取代
- ▶ 提示到这里了，可以再想一下
- ▶ 记 $g[u][i][ku=0/1][kt=0/1] = f[fa(u,i)][kt] - f[u][ku]$
- ▶ 其中 $fa(u,i)$ 表示深度比 u 小 2^i 的祖先
- ▶ 这个倍增数组可以轻松预处理
- ▶ 那么每次拿到 a, b ，求出 LCA c ，通过倍增的方式求出 a 和 b 到 c 的两个儿子这条链上满足要求的最小代价，倍增的细节画图解释
- ▶ 最后枚举这两个儿子取不取，枚举 c 取不取，统一取 $\min$

NOIP2018 保卫王国

- ▶ 都想到树链剖分了，事实上，在这种不带修改只查询的情况下，很多时候树链剖分都能被倍增取代
- ▶ 提示到这里了，可以再想一下
- ▶ 记 $g[u][i][ku=0/1][kt=0/1] = f[fa(u,i)][kt] - f[u][ku]$
- ▶ 其中 $fa(u,i)$ 表示深度比 u 小 2^i 的祖先
- ▶ 这个倍增数组可以轻松预处理
- ▶ 那么每次拿到 a, b ，求出 LCA c ，通过倍增的方式求出 a 和 b 到 c 的两个儿子这条链上满足要求的最小代价，倍增的细节画图解释
- ▶ 最后枚举这两个儿子取不取，枚举 c 取不取，统一取 $\min$
- ▶ 事实上你还得开一个 $h[u][0/1]$ 记录一下 u 不取/取时， u 子树以外的部分的最小代价，最后的答案还得考虑进 $h[c][0/1]$

ZJOI2007 时态同步

小Q在电子工艺实习课上学习焊接电路板。一块电路板由若干个元件组成，我们不妨称之为节点，并将其用数字 $1, 2, 3 \dots$ 进行标号。电路板的各个节点由若干不相交的导线相连接，且对于电路板的任何两个节点，都存在且仅存在一条通路（通路指连接两个元件的导线序列）。

在电路板上存在一个特殊的元件称为“激发器”。当激发器工作后，产生一个激励电流，通过导线传向每一个它所连接的节点。而中间节点接收到激励电流后，得到信息，并将该激励电流传向与它连接并且尚未接收到激励电流的节点。最终，激励电流将到达一些“终止节点”——接收激励电流之后不再转发的节点。

激励电流在导线上的传播是需要花费时间的，对于每条边 e ，激励电流通过它需要的时间为 t_e ，而节点接收到激励电流后的转发可以认为是在瞬间完成的。现在这块电路板要求每一个“终止节点”同时得到激励电路——即保持时态同步。由于当前的构造并不符合时态同步的要求，故需要通过改变连接线的构造。目前小Q有一个道具，使用一次该道具，可以使得激励电流通过某条连接导线的时间增加一个单位。请问小Q最少使用多少次道具才可使得所有的“终止节点”时态同步？

对于40%的数据， $N \leq 1000$

对于100%的数据， $N \leq 500000$

对于所有的数据， $t_e \leq 1000000$

ZJOI2007 时 态 同 步

- ▶ 简化题意：
- ▶ 给出一棵边有长度的有根树，花费1的代价可以将任意一条边长增大1，求使得所有叶子到根距离相等的最小花费。

ZJOI2007 时 态 同 步

- ▶ 简化题意：
- ▶ 给出一棵边有长度的有根树，花费1的代价可以将任意一条边长增大1，求使得所有叶子到根距离相等的最小花费。
- ▶ 考虑这样一个结论：若所有叶子到根距离相等，则任意一棵子树的所有叶子到子树根距离相等

ZJOI2007 时 态 同 步

- ▶ 简化题意：
- ▶ 给出一棵边有长度的有根树，花费1的代价可以将任意一条边长增大1，求使得所有叶子到根距离相等的最小花费。
- ▶ 考虑这样一个结论：若所有叶子到根距离相等，则任意一棵子树的所有叶子到子树根距离相等
- ▶ 然后与其说是树形DP，不如说是个贪心

ZJOI2007 时 态 同 步

- ▶ 简化题意：
- ▶ 给出一棵边有长度的有根树，花费1的代价可以将任意一条边长增大1，求使得所有叶子到根距离相等的最小花费。
- ▶ 考虑这样一个结论：若所有叶子到根距离相等，则任意一棵子树的所有叶子到子树根距离相等
- ▶ 然后与其说是树形DP，不如说是个贪心
- ▶ 自底向上对每一棵子树进行处理，找到子树根到叶子的最大距离，将其他与根相连的边增长到符合要求，然后记录下这个距离用于更上一层的子树处理即可

ZJOI2007 时 态 同 步

- ▶ 简化题意：
- ▶ 给出一棵边有长度的有根树，花费1的代价可以将任意一条边长增大1，求使得所有叶子到根距离相等的最小花费。
- ▶ 考虑这样一个结论：若所有叶子到根距离相等，则任意一棵子树的所有叶子到子树根距离相等
- ▶ 然后与其说是树形DP，不如说是个贪心
- ▶ 自底向上对每一棵子树进行处理，找到子树根到叶子的最大距离，将其他与根相连的边增长到符合要求，然后记录下这个距离用于更上一层的子树处理即可
- ▶ 我觉得画图会好讲很多

CSP-S2019 Emiya 家今天的饭

Emiya 是个擅长做菜的高中生，他共掌握 n 种**烹饪方法**，且会使用 m 种**主要食材**做菜。为了方便叙述，我们对烹饪方法从 $1 \sim n$ 编号，对主要食材从 $1 \sim m$ 编号。

Emiya 做的每道菜都将使用**恰好一种**烹饪方法与**恰好一种**主要食材。更具体地，Emiya 会做 $a_{i,j}$ 道不同的使用烹饪方法 i 和主要食材 j 的菜 ($1 \leq i \leq n, 1 \leq j \leq m$)，这也意味着 Emiya 总共会做 $\sum_{i=1}^n \sum_{j=1}^m a_{i,j}$ 道不同的菜。

Emiya 今天要准备一桌饭招待 Yazid 和 Rin 这对好朋友，然而三个人对菜的搭配有不同的要求，更具体地，对于一种包含 k 道菜的搭配方案而言：

- Emiya 不会让大家饿肚子，所以将做**至少一道菜**，即 $k \geq 1$
- Rin 希望品尝不同烹饪方法做出的菜，因此她要求每道菜的**烹饪方法互不相同**
- Yazid 不希望品尝太多同一食材做出的菜，因此他要求每种**主要食材**至多在一半的菜（即 $\lfloor \frac{k}{2} \rfloor$ 道菜）中被使用

这里的 $\lfloor x \rfloor$ 为下取整函数，表示不超过 x 的最大整数。

这些要求难不倒 Emiya，但他想知道共有多少种不同的符合要求的搭配方案。两种方案不同，当且仅当存在至少一道菜在一种方案中出现，而不在另一种方案中出现。

Emiya 找到了你，请你帮他计算，你只需要告诉他符合所有要求的搭配方案数对质数 998,244,353 取模的结果。

对于所有测试点，保证 $1 \leq n \leq 100$, $1 \leq m \leq 2000$, $0 \leq a_{i,j} < 998,244,353$ 。

CSP-S2019 Emiya 家今天的饭

- ▶ 这居然是CSP-S2019的D2T1? 比起传统的签到题确实难太多了
- ▶ 简化题意:
- ▶ 给出一个矩阵, 全局至少选一个格子, 每行最多选一个格子, 每列选的格子数不超过全局选格子数的一半, 求所有选格子方法中格子权值之积的总和。

CSP-S2019 Emiya 家今天的饭

- ▶ 这居然是CSP-S2019的D2T1? 比起传统的签到题确实难太多了
- ▶ 简化题意:
- ▶ 给出一个矩阵, 全局至少选一个格子, 每行最多选一个格子, 每列选的格子数不超过全局选格子数的一半, 求所有选格子方法中格子权值之积的总和。
- ▶ 乍一看很难, 但如果删去第三个条件, 题目将变得异常简单。
- ▶ 行与行之间的选择互不影响 (乘法原理), 行内的选择互相排斥 (加法原理), 再排除掉每一行都不选的情况, 答案就是(每行权值和+1)的乘积-1

CSP-S2019 Emiya 家今天的饭

- ▶ 这居然是CSP-S2019的D2T1? 比起传统的签到题确实难太多了
- ▶ 简化题意:
- ▶ 给出一个矩阵, 全局至少选一个格子, 每行最多选一个格子, 每列选的格子数不超过全局选格子数的一半, 求所有选格子方法中格子权值之积的总和。
- ▶ 乍一看很难, 但如果删去第三个条件, 题目将变得异常简单。
- ▶ 行与行之间的选择互不影响 (乘法原理), 行内的选择互相排斥 (加法原理), 再排除掉每一行都不选的情况, 答案就是(每行权值和+1)的乘积-1
- ▶ 再看一眼第三个条件, 我们发现, 如果某列选的格子数超过全局选格子数的一半, 那么其他列都可以随意选, 因为不可能有两列都超过一半。
- ▶ 于是问题转化为, 求所有某一列超过一半的方案中格子权值乘积之和。
- ▶ 用不考虑第三个条件的答案减去这一部分既是原问题的答案

CSP-S2019 Emiya 家今天的饭

- ▶ 一个很正常的想法——枚举哪一列超过了一半
- ▶ 那么只要确定那些行选该列，哪些行不选，哪些行选其他列，就可以统计这一情况下对答案的贡献

CSP-S2019 Emiya 家今天的饭

- ▶ 一个很正常的想法——枚举哪一列超过了一半
- ▶ 那么只要确定那些行选该列，哪些行不选，哪些行选其他列，就可以统计这一情况下对答案的贡献
- ▶ 我们有一个朴素的想法，用 $f[i][j][k]$ 表示处理到第 i 行，本列（记为第 p 列）选了 j 个，总共选了 k 个的方案数
- ▶ 不难得出 $f[i][j][k] = f[i-1][j][k]$ (都不选) + $f[i-1][j-1][k-1] * a[i][p]$ (选本列) + $f[i-1][j][k-1] * (\text{sum}[i] - a[i][p])$ (选其他列)

CSP-S2019 Emiya 家今天的饭

- ▶ 一个很正常的想法——枚举哪一列超过了一半
- ▶ 那么只要确定那些行选该列，哪些行不选，哪些行选其他列，就可以统计这一情况下对答案的贡献
- ▶ 我们有一个朴素的想法，用 $f[i][j][k]$ 表示处理到第 i 行，本列（记为第 p 列）选了 j 个，总共选了 k 个的方案数
- ▶ 不难得出 $f[i][j][k] = f[i-1][j][k]$ (都不选) + $f[i-1][j-1][k-1] * a[i][p]$ (选本列) + $f[i-1][j][k-1] * (\text{sum}[i] - a[i][p])$ (选其他列)
- ▶ 然而这个状态数是 $O(n^3)$ 的，会超时

CSP-S2019 Emiya 家今天的饭

- ▶ 我们转变一下思路，事实上，我们并不关心总共选了多少个，我们反而更关心是否有 $j > k/2$ ，也就是我们只希望保证 $2j - k > 0$

CSP-S2019 Emiya 家今天的饭

- ▶ 我们转变一下思路，事实上，我们并不关心总共选了多少个，我们反而更关心是否有 $j > k/2$ ，也就是我们只希望保证 $2j - k > 0$
- ▶ 事实上，我们只需要记录 $2j - k$ 的值，因为该值如何得出对后续决策没有影响
- ▶ 当接下来的一行选择 p 列时， $(2j - k)++$ ；选择 p 列以外时， $(2j - k)--$ ；都不选时， $2j - k$ 不变

CSP-S2019 Emiya 家今天的饭

- ▶ 我们转变一下思路，事实上，我们并不关心总共选了多少个，我们反而更关心是否有 $j > k/2$ ，也就是我们只希望保证 $2j - k > 0$
- ▶ 事实上，我们只需要记录 $2j - k$ 的值，因为该值如何得出对后续决策没有影响
- ▶ 当接下来的一行选择 p 列时， $(2j - k)++$ ；选择 p 列以外时， $(2j - k)--$ ；都不选时， $2j - k$ 不变
- ▶ 因此用 $f[i][x]$ 来记录处理了前 i 行， $x = 2j - k$ 的方案数，由于 $2j - k \in [-n, n]$ ，可以对 x 这一位做 $+n$ 的偏移， $f[i][n]$ 成为原来的 $f[i][0]$ ， $f[i][0]$ 成为原来的 $f[i][-n]$
- ▶ $f[i][j][x] = f[i-1][x] (\text{都不选}) + f[i-1][x-1] * a[i][p] (\text{选本列}) + f[i-1][x+1] * (\text{sum}[i] - a[i][p]) (\text{选其他列})$
- ▶ 边界条件为 $f[0][n] = 1$ ， $f[0][\text{其余}] = 0$
- ▶ 对于一列超过一半的情况可以 $O(n^2)$ 解决，总复杂度 $O(mn^2)$

P1156 垃圾陷阱

卡门——农夫约翰极其珍视的一条 `Holsteins` 奶牛——已经落到了“垃圾井”中。“垃圾井”是农夫们扔垃圾的地方，它的深度为 D ($2 \leq D \leq 100$) 英尺。

卡门想把垃圾堆起来，等到堆得与井同样高时，她就能逃出井外了。另外，卡门可以通过吃一些垃圾来维持自己的生命。

每个垃圾都可以用来吃或堆放，并且堆放垃圾不用花费卡门的时间。

假设卡门预先知道了每个垃圾扔下的时间 t ($0 < t \leq 1000$)，以及每个垃圾堆放的高度 h ($1 \leq h \leq 25$) 和吃进该垃圾能维持生命的时间 f ($1 \leq f \leq 30$)，要求出卡门最早能逃出井外的时间，假设卡门当前体内有足够持续10小时的能量，如果卡门10小时内没有进食，卡门就将饿死。

P1156 垃圾陷阱

- ▶ 题目题意需要好好理解

P1156 垃圾陷阱

- ▶ 题目题意需要好好理解
- ▶ 题目其实具有背包的特征
- ▶ 考虑一下，其实所有垃圾维持时长总和是一定的，你可以把一部分垃圾用来填埋以产生一个负的收益
- ▶ 现在其实是在把坑（背包）填满的情况下，负收益尽可能小

P1156 垃圾陷阱

- ▶ 题目题意需要好好理解
- ▶ 题目其实具有背包的特征
- ▶ 考虑一下，其实所有垃圾维生时长总和是一定的，你可以把一部分垃圾用来填埋以产生一个负的收益
- ▶ 现在其实是要求在把坑（背包）填满的情况下，负收益尽可能小
- ▶ 对垃圾按时间排序
- ▶ $dp[i][j]$ 表示前 i 个垃圾落下后，填到 j 的高度所能获得的最大维生时间
- ▶ 如果吃， $dp[i][j] + f[i+1] \rightarrow dp[i+1][j]$ ；如果不吃， $dp[i][j] \rightarrow dp[i+1][j+h[i+1]]$

P1156 垃圾陷阱

- ▶ 题目题意需要好好理解
- ▶ 题目其实具有背包的特征
- ▶ 考虑一下，其实所有垃圾维生时长总和是一定的，你可以把一部分垃圾用来填埋以产生一个负的收益
- ▶ 现在其实是在把坑（背包）填满的情况下，负收益尽可能小
- ▶ 对垃圾按时间排序
- ▶ $dp[i][j]$ 表示前 i 个垃圾落下后，填到 j 的高度所能获得的最大维生时间
- ▶ 如果吃， $dp[i][j] + f[i+1] \rightarrow dp[i+1][j]$ ；如果不吃， $dp[i][j] \rightarrow dp[i+1][j+h[i+1]]$
- ▶ 就是个01背包，要注意，所有 $dp[i][j] < t[i+1]$ 的部分不能用来更新 $dp[i+1][...]$

P2112 鸿雁传书

题目背景

展开

小明给小红写了一封情书，他想把文章变得更完美，所以要进行排版。

题目描述

他一共写了 N 个单词，为了美观，要把 N 个单词分成 K 行。单词的相对顺序不能变化。为了简化问题，无需考虑单词间的空格。

小红会喜欢整齐的情书，小明想赢得小红的芳心，所以，他找到你，想让你帮他写一个程序，帮他排版，使得每行字母数的方差最小。请你求出最小的方差。

【数据范围】

对于30%数据， $N \leq 100, K \leq 3$ 。

为了方便，把这里的所有 K 改成 M

对于全部数据， $N \leq 1000, K \leq 100$. 单词长度 ≤ 20 .

P2112 鸿雁传书

- ▶ 单词的总长是固定的，行数也是固定的，因此平均数是固定的，记为 x
- ▶ 记每行长度为 $l[i]$ ，则题目既是要最小化 $\sum (l[i]-x)^2$

P2112 鸿雁传书

- ▶ 单词的总长是固定的，行数也是固定的，因此平均数是固定的，记为 x
- ▶ 记每行长度为 $l[i]$ ，则题目既是要最小化 $\sum(l[i]-x)^2$
- ▶ 假设前 i 用了前 j 个单词，则这些单词的分割方法不会对后续决策产生影响
- ▶ 用 $f[i][j]$ 表示前 i 行恰好填了前 j 个单词，所产生的最小的 $\sum(l[k]-x)^2$ ($k = 1 \rightarrow i$)
- ▶ 那么枚举分割点 k ，则 $f[i][j] = \min\{f[i-1][k] + (\text{sum}[j] - \text{sum}[k] - x)^2 \mid k = 0..j\}$

P2112 鸿雁传书

- ▶ 单词的总长是固定的，行数也是固定的，因此平均数是固定的，记为 x
- ▶ 记每行长度为 $l[i]$ ，则题目既是要最小化 $\sum(l[i]-x)^2$
- ▶ 假设前 i 用了前 j 个单词，则这些单词的分割方法不会对后续决策产生影响
- ▶ 用 $f[i][j]$ 表示前 i 行恰好填了前 j 个单词，所产生的最小的 $\sum(l[k]-x)^2$ ($k = 1 \rightarrow i$)
- ▶ 那么枚举分割点 k ，则 $f[i][j] = \min\{f[i-1][k] + (\text{sum}[j] - \text{sum}[k] - x)^2 \mid k = 0..j\}$
- ▶ $O(n^2m)$ 理论上会超时，实际上能水过

P2112 鸿雁传书

- ▶ 单词的总长是固定的，行数也是固定的，因此平均数是固定的，记为 x
- ▶ 记每行长度为 $l[i]$ ，则题目既是要最小化 $\sum(l[i]-x)^2$
- ▶ 假设前 i 用了前 j 个单词，则这些单词的分割方法不会对后续决策产生影响
- ▶ 用 $f[i][j]$ 表示前 i 行恰好填了前 j 个单词，所产生的最小的 $\sum(l[k]-x)^2$ ($k = 1 \rightarrow i$)
- ▶ 那么枚举分割点 k ，则 $f[i][j] = \min\{f[i-1][k] + (\text{sum}[j] - \text{sum}[k] - x)^2 \mid k = 0..j\}$
- ▶ $O(n^2m)$ 理论上会超时，实际上能水过
- ▶ 仔细想一下，这个分割点的位置是满足决策单调性的，假如 $f[i][j]$ 的最优决策分割点在 k_0 ， $f[i+1][j]$ 的最优分割点在 k_1 ，一定有 $k_0 \leq k_1$ ； $f[i][j+1]$ 的最优分割点在 k_2 ，一定有 $k_0 \leq k_2$
- ▶ 凭直觉应该是对的，我没证明，大不了对拍一下

HNOI2004 打鼹鼠

鼹鼠是一种很喜欢挖洞的动物，但每过一定的时间，它还是喜欢把头探出到地面上来透透气的。根据这个特点阿牛编写了一个打鼹鼠的游戏：在一个 $n*n$ 的网格中，在某些时刻鼹鼠会在某一个网格探出头来透透气。你可以控制一个机器人来打鼹鼠，如果 i 时刻鼹鼠在某个网格中出现，而机器人也处于同一网格的话，那么这个鼹鼠就会被机器人打死。而机器人每一时刻只能够移动一格或停留在原地不动。机器人的移动是指从当前所处的网格移向相邻的网格，即从坐标为 (i,j) 的网格移向 $(i-1,j),(i+1,j),(i,j-1),(i,j+1)$ 四个网格，机器人不能走出整个 $n*n$ 的网格。游戏开始时，你可以自由选定机器人的初始位置。

现在知道在一段时间内，鼹鼠出现的时间和地点，请编写一个程序使机器人在这一段时间内打死尽可能多的鼹鼠。

n ($n \leq 1000$) , m ($m \leq 10000$) ,

HNOI2004 打鼹鼠

- ▶ 这个题目非常精妙
- ▶ 网格其实是误导信息，机器人在网格的哪里其实并不关键

HNOI2004 打鼹鼠

- ▶ 这个题目非常精妙
- ▶ 网格其实是误导信息，机器人在网格的哪里其实并不关键
- ▶ 我们换一个角度思考，假设我们已经确定了要打哪些鼹鼠，如何验证这份打鼹鼠计划是否能实现

HNOI2004 打鼯鼠

- ▶ 这个题目非常精妙
- ▶ 网格其实是误导信息，机器人在网格的哪里其实并不关键
- ▶ 我们换一个角度思考，假设我们已经确定了要打哪些鼯鼠，如何验证这份打鼯鼠计划是否能实现
- ▶ 只需要验证相邻次序出现的两只鼯鼠，其曼哈顿距离是否不大于时间差即可

HNOI2004 打鼯鼠

- ▶ 这个题目非常精妙
- ▶ 网格其实是误导信息，机器人在网格的哪里其实并不关键
- ▶ 我们换一个角度思考，假设我们已经确定了要打哪些鼯鼠，如何验证这份打鼯鼠计划是否能实现
- ▶ 只需要验证相邻次序出现的两只鼯鼠，其曼哈顿距离是否不大于时间差即可
- ▶ 所以对鼯鼠按出现时间排序， $dp[i]$ 表示，一定要打第 i 只鼯鼠的情况下，前 i 只鼯鼠最多打到多少只
- ▶ 则 $dp[i] = \max\{1, dp[j] + 1 \mid \text{dis}(i, j) \leq t[i] - t[j], j = 1, 2, \dots, i-1\}$
- ▶ $O(nm)$ 解决本题

P6371 V

题目描述

使用给定的数字，组成一些在 $[A, B]$ 之间的数使得这些数每个都能被 X 整除。

输入格式

输入第一行包含三个整数 X, A, B 。

第二行为一个数字串，表示可以使用的数字。数字不会重复出现。

输出格式

输出一行一个整数，表示在 $[A, B]$ 这个区间用这些给定的数字能组成多少个被 X 整除的数字。

数据规模与约定

- 对于 100% 的数据，保证 $1 \leq X < 10^{11}$, $1 \leq A \leq B < 10^{11}$ 。

 展开

P6371 V

- ▶ 题意很简单，摆明了告诉你数位DP

P6371 V

- ▶ 题意很简单，摆明了告诉你数位DP
- ▶ 拿这题介绍一下数位DP的基本思路
- ▶ 给出了 $[A,B]$ 这一区间，可以将问题转化为：
- ▶ 求不大于B的满足要求的个数减去不大于A的满足要求的个数

P6371 V

- ▶ 题意很简单，摆明了告诉你数位DP
- ▶ 拿这题介绍一下数位DP的基本思路
- ▶ 给出了 $[A,B]$ 这一区间，可以将问题转化为：
- ▶ 求不大于B的满足要求的个数减去不大于A的满足要求的个数
- ▶ 数位DP的一般思路在于，按一定顺序对每一个数位进行处理，同时把一些和答案相关的统计信息计入状态，其中较为通用的两个统计信息是：

P6371 V

- ▶ 题意很简单，摆明了告诉你数位DP
- ▶ 拿这题介绍一下数位DP的基本思路
- ▶ 给出了 $[A,B]$ 这一区间，可以将问题转化为：
- ▶ 求不大于B的满足要求的个数减去不大于A的满足要求的个数
- ▶ 数位DP的一般思路在于，按一定顺序对每一个数位进行处理，同时把一些和答案相关的统计信息计入状态，其中较为通用的两个统计信息是：
 - ▶ 1. 前 i 位是否和上界的前 i 位相等（如果相等则后一位枚举时不可超过上界）
 - ▶ 2. 前 i 位是否为前导零（前导0不算使用了0）

P6371 V

- ▶ 题意很简单，摆明了告诉你数位DP
- ▶ 拿这题介绍一下数位DP的基本思路
- ▶ 给出了 $[A,B]$ 这一区间，可以将问题转化为：
- ▶ 求不大于B的满足要求的个数减去不大于A的满足要求的个数
- ▶ 数位DP的一般思路在于，按一定顺序对每一个数位进行处理，同时把一些和答案相关的统计信息计入状态，其中较为通用的两个统计信息是：
 - ▶ 1. 前 i 位是否和上界的前 i 位相等（如果相等则后一位枚举时不可超过上界）
 - ▶ 2. 前 i 位是否为前导零（前导0不算使用了0）
- ▶ 例如本题 $dp[i][j][lim][zero]$ ，其中 j 表示的是前 i 位对 X 的余数

P6371 V

- ▶ 转移并不难

P6371 V

- ▶ 转移并不难
- ▶ 假设我要求不大于A的个数
- ▶ 对于 $\text{lim}=1$ 时（即前几位和A前几位相同），下一位也最多枚举到与A相同，否则可以枚举到9，如果这一位不再和A相同，则转移到 $\text{lim}=0$
- ▶ 如果之前有前导零，这一位不是0，则转移到 $\text{zero}=0$ ，否则继续转移到 $\text{zero}=1$
- ▶ 除了前导零的情况，都要排除掉不能被选用的数字

P6371 V

- ▶ 转移并不难
- ▶ 假设我要求不大于A的个数
- ▶ 对于 $lim==1$ 时（即前几位和A前几位相同），下一位也最多枚举到与A相同，否则可以枚举到9，如果这一位不再和A相同，则转移到 $lim=0$
- ▶ 如果之前有前导零，这一位不是0，则转移到 $zero=0$ ，否则继续转移到 $zero=1$
- ▶ 除了前导零的情况，都要排除掉不能被选用的数字
- ▶ 具体转移式
- ▶ $dp[i][j][0][0] \rightarrow dp[i-1][(j+k*10^{(i-1)})\%X][0][0]$ k可用
- ▶ $dp[i][j][1][0] \rightarrow dp[i-1][(j+k*10^{(i-1)})\%X][0][0]$ $k < A[i-1]$ ，k可用
- ▶ $dp[i][j][1][0] \rightarrow dp[i-1][(j+A[i-1]*10^{(i-1)})\%X][1][0]$ $A[i-1]$ 可用
- ▶ $dp[i][0][?][1] \rightarrow dp[i-1][0][?][1]$ lim 的情况自己判断
- ▶ $dp[i][0][?][1] \rightarrow dp[i-1][k*10^{(i-1)}\%X][?][0]$ lim 的情况自己判断，k可用

P6371 V

- ▶ 最后复杂度 $O(\text{len}(B) * X * 10)$

P6371 V

- ▶ 最后复杂度 $O(\text{len}(B) * X * 10)$
- ▶ 但是 X 最多 10^{11} ，炸了

P6371 V

- ▶ 最后复杂度 $O(\text{len}(B) * X * 10)$
- ▶ 但是 X 最多 10^{11} ，炸了
- ▶ 其实仔细想想，若 $X > 10^5$ ，那么最多有 $B/X \leq 10^6$ 个满足要求的数，于是暴力枚举 X 的倍数判断即可

NOI2006 网络收费

题目描述

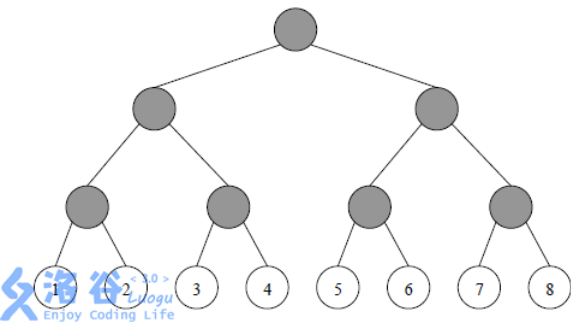
网络已经成为当今世界不可或缺的一部分。每天都有数以亿计的人使用网络进行学习、科研、娱乐等活动。然而，不可忽视的一点就是网络本身有着庞大的运行费用。所以，向使用网络的人进行适当的收费是必须的，也是合理的。

MY 市 NS 中学就有着这样一个教育网络。网络中的用户一共有 2^N 个，编号依次为 $1, 2, 3, \dots, 2^N$ 。这些用户之间是用路由点和网线组成的。用户、路由点与网线共同构成一个满二叉树结构。树中的每一个叶子结点都是一个用户，每一个非叶子结点（灰色）都是一个路由点，而每一条边都是一条网线（见下图，用户结点中的数字为其编号）。

MY 网络公司的网络收费方式比较奇特，称为“配对收费”。即对于每两个用户 $i, j (1 \leq i < j \leq 2^N)$ 进行收费。由于用户可以自行选择两种付费方式 A、B 中的一种，所以网络公司向学校收取的费用与每一位用户的付费方式有关。该费用等于每两位不同用户配对产生费用之和。

为了描述方便，首先定义这棵网络树上的一些概念：

- 祖先：根结点没有祖先，非根结点的祖先包括它的父亲以及它的父亲的祖先；
- 管辖叶结点：叶结点本身不管辖任何叶结点，非叶结点管辖它的左儿子所管辖的叶结点与它的右儿子所管辖的叶结点；
- 距离：在树上连接两个点之间的用边最少的路径所含的边数。



对于任两个用户 $i, j (1 \leq i < j \leq 2^N)$ ，首先在树上找到与它们距离最近的公共祖先：路由点 P ，然后观察 P 所管辖的叶结点（即用户）中选择付费方式 A 与 B 的人数，分别记为 n_A 与 n_B ，接着按照网络管理条例第 X 章第 Y 条第 Z 款进行收费（如下表），其中 $F_{i,j}$ 为 i 和 j 之间的流量，且为已知量。

i 付费方式	j 付费方式	n_A 与 n_B 大小关系	付费系数 k	实际付费
A	A	$n_A < n_B$	2	$k * F_{i,j}$
A	B		1	
B	A		1	
B	B		0	
A	A	$n_A \geq n_B$	0	
A	B		1	
B	A		1	
B	B		2	

由于最终所付费用与付费方式有关，所以 NS 中学的用户希望能够自行改变自己的付费方式以减少总付费。然而，由于网络公司已经将每个用户注册时所选择的付费方式记录在案，所以对于用户 i ，如果他/她想改变付费方式（由 A 改为 B 或由 B 改为 A），就必须支付 C_i 元给网络公司以修改档案（修改付费方式记录）。

现在的问题是，给定每个用户注册时所选择的付费方式以及 C_i ，试求这些用户应该如何选择自己的付费方式以使得 NS 中学支付给网络公司的总费用最少（更改付费方式费用+配对收费的费用）。

NOI2006 网络收费

- ▶ 简化题意：
- ▶ 给出一棵满二叉树，叶子节点有黑白两色，对于任意两个叶子节点 i, j ，如果颜色不同，则产生 $F[i][j]$ 的代价；若颜色相同，且颜色占 $LCA(i, j)$ 为根的子树叶子节点的一半以上，则无需代价，否则需要支付 $2F[i][j]$ 的代价
- ▶ 现在可以花费 $C[i]$ 的代价将叶子 i 反色，求总代价的最小值

NOI2006 网络收费

- ▶ 简化题意：
- ▶ 给出一棵满二叉树，叶子节点有黑白两色，对于任意两个叶子节点 i, j ，如果颜色不同，则产生 $F[i][j]$ 的代价；若颜色相同，且颜色占 $LCA(i, j)$ 为根的子树叶子节点的一半以上，则无需代价，否则需要支付 $2F[i][j]$ 的代价
- ▶ 现在可以花费 $C[i]$ 的代价将叶子 i 反色，求总代价的最小值
- ▶ 这个收费方式相当奇怪，在点对之间的计价似乎很难逃脱出 $O((2^n)^2)$ 的复杂度，这使得后续处理变得艰难

NOI2006 网络收费

- ▶ 简化题意：
- ▶ 给出一棵满二叉树，叶子节点有黑白两色，对于任意两个叶子节点 i, j ，如果颜色不同，则产生 $F[i][j]$ 的代价；若颜色相同，且颜色占 $LCA(i, j)$ 为根的子树叶子节点的一半以上，则无需代价，否则需要支付 $2F[i][j]$ 的代价
- ▶ 现在可以花费 $C[i]$ 的代价将叶子 i 反色，求总代价的最小值
- ▶ 这个收费方式相当奇怪，在点对之间的计价似乎很难逃脱出 $O((2^n)^2)$ 的复杂度，这使得后续处理变得艰难
- ▶ 思考这样一个问题，这个2/1/0的系数本质上是什么，似乎是 i, j 两个叶子中和 LCA 子树色调（即超过一半的颜色书）不同的个数

NOI2006 网络收费

- ▶ 简化题意：
- ▶ 给出一棵满二叉树，叶子节点有黑白两色，对于任意两个叶子节点 i, j ，如果颜色不同，则产生 $F[i][j]$ 的代价；若颜色相同，且颜色占 $LCA(i, j)$ 为根的子树叶子节点的一半以上，则无需代价，否则需要支付 $2F[i][j]$ 的代价
- ▶ 现在可以花费 $C[i]$ 的代价将叶子 i 反色，求总代价的最小值
- ▶ 这个收费方式相当奇怪，在点对之间的计价似乎很难逃脱出 $O((2^n)^2)$ 的复杂度，这使得后续处理变得艰难
- ▶ 思考这样一个问题，这个2/1/0的系数本质上是什么，似乎是 i, j 两个叶子中和 LCA 子树色调（即超过一半的颜色书）不同的个数
- ▶ 所以，我们定义一个中间节点的色调为其子树中较多的颜色的色调，那么每个和色调不同的叶子节点都会产生1的贡献（要乘上它和另一分支的 ΣF ）

NOI2006 网络收费

- ▶ 于是我们想到了这样的思路
- ▶ $dp[u][w]$ 表示, u 为根的子树, 有 w 个叶子节点是白色的最小代价

NOI2006 网络收费

- ▶ 于是我们想到了这样的思路
- ▶ $dp[u][w]$ 表示, u 为根的子树, 有 w 个叶子节点是白色的最小代价
- ▶ 但当你递归到叶子节点时, 你会发现你需要知道你的祖先链的色调情况才能计算这个叶子对代价的贡献

NOI2006 网络收费

- ▶ 于是我们想到了这样的思路
- ▶ $dp[u][w]$ 表示, u 为根的子树, 有 w 个叶子节点是白色的最小代价
- ▶ 但当你递归到叶子节点时, 你会发现你需要知道你的祖先链的色调情况才能计算这个叶子对代价的贡献
- ▶ 所以我们要把祖先链的色调计入状态, 显然这是可以状压的

NOI2006 网络收费

- ▶ 于是我们想到了这样的思路
- ▶ $dp[u][w]$ 表示, u 为根的子树, 有 w 个叶子节点是白色的最小代价
- ▶ 但当你递归到叶子节点时, 你会发现你需要知道你的祖先链的色调情况才能计算这个叶子对代价的贡献
- ▶ 所以我们要把祖先链的色调计入状态, 显然这是可以状压的
- ▶ $dp[u][w][S]$ 表示 u 为根的子树, 有 w 个叶子节点是白色, 且其祖先中深度为 i 的节点色调为 S 的 2^i 二进制位的最小代价

NOI2006 网络收费

- ▶ 于是我们想到了这样的思路
- ▶ $dp[u][w]$ 表示, u 为根的子树, 有 w 个叶子节点是白色的最小代价
- ▶ 但当你递归到叶子节点时, 你会发现你需要知道你的祖先链的色调情况才能计算这个叶子对代价的贡献
- ▶ 所以我们要把祖先链的色调计入状态, 显然这是可以状压的
- ▶ $dp[u][w][S]$ 表示 u 为根的子树, 有 w 个叶子节点是白色, 且其祖先中深度为 i 的节点色调为 S 的 2^i 二进制位的最小代价
- ▶ 转移是枚举 w 个白色节点在左右子树的分配, 注意这里枚举带来的复杂度是 $O(n \cdot 2^n)$ (不考虑 S)

NOI2006 网络收费

- ▶ 于是我们想到了这样的思路
- ▶ $dp[u][w]$ 表示, u 为根的子树, 有 w 个叶子节点是白色的最小代价
- ▶ 但当你递归到叶子节点时, 你会发现你需要知道你的祖先链的色调情况才能计算这个叶子对代价的贡献
- ▶ 所以我们要把祖先链的色调计入状态, 显然这是可以状压的
- ▶ $dp[u][w][S]$ 表示 u 为根的子树, 有 w 个叶子节点是白色, 且其祖先中深度为 i 的节点色调为 S 的 2^i 二进制位的最小代价
- ▶ 转移是枚举 w 个白色节点在左右子树的分配, 注意这里枚举带来的复杂度是 $O(n \cdot 2^n)$ (不考虑 S)
- ▶ 叶子节点对应 S 所产生的代价可以提前预处理
- ▶ 总复杂度 $O(n \cdot 2^{2n})$